

Budget Tracker - Full Stack Application

Deployment & System Test Documentation

Course: CSCE 548 - Full Stack Development

Project: Complete N-Tier Application (Projects 1-4)

Technologies: React, Flask, Python, MySQL, Railway, Render, Netlify

Table of Contents

1. [Project Overview](#)
 2. [Architecture](#)
 3. [Prerequisites](#)
 4. [Database Setup \(Railway\)](#)
 5. [Backend Deployment \(Render\)](#)
 6. [Frontend Deployment \(Netlify\)](#)
 7. [Local Development Setup](#)
 8. [Testing & Verification](#)
 9. [AI-Generated Code Analysis](#)
 10. [Troubleshooting](#)
 11. [Project Statistics](#)
 12. [Credits](#)
-

Project Overview

Budget Tracker is a complete full-stack web application for personal budget management. The application demonstrates a professional n-tier architecture with:

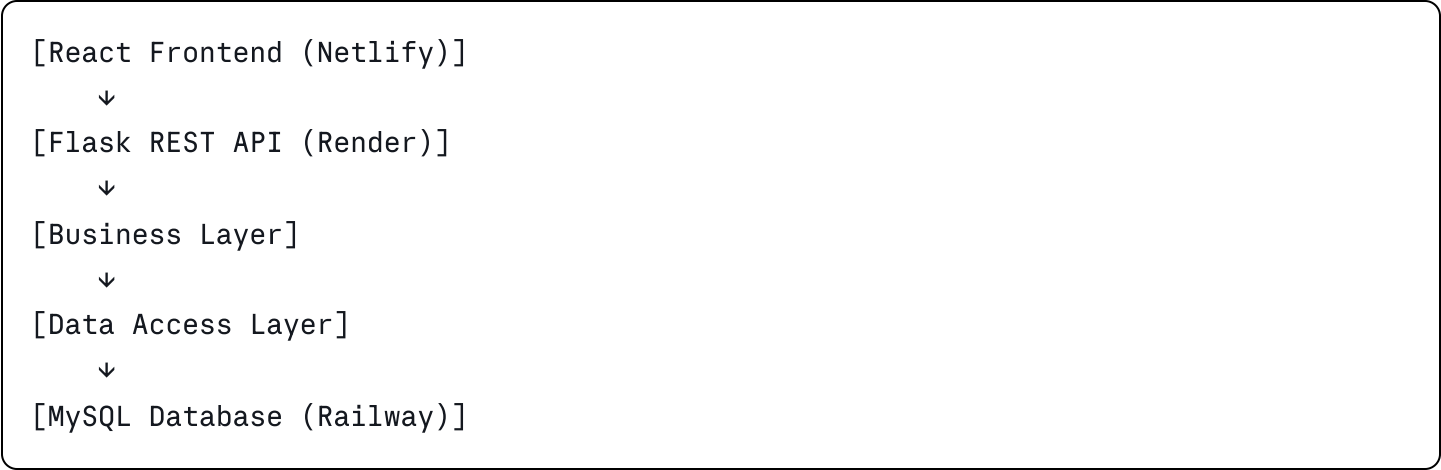
- **Data Layer:** MySQL database with 5 normalized tables
- **Business Layer:** Python business logic with validation rules
- **Service Layer:** Flask REST API with CRUD endpoints
- **Client Layer:** React single-page application with purple theme

Key Features

- **Full CRUD Operations** for all 5 tables (Users, Categories, Budgets, Budget Rules, Transactions)
 - **Business Rule Validation** (unique constraints, date validation, spending limits)
 - **Subset Queries** (filter budgets by user, rules by budget, transactions by user)
 - **Responsive Design** with purple ( #A020F0) theme
 - **Cloud Deployment** (Railway + Render + Netlify)
-

Architecture

System Diagram



Technology Stack

Layer	Technology	Hosting
Frontend	React 18.2, Axios, React Router	Netlify
Service	Flask, Flask-CORS, Gunicorn	Render.com
Business	Python 3.x	(part of backend)
Data Access	mysql-connector-python	(part of backend)
Database	MySQL 8.0	Railway

Prerequisites

Before deploying, ensure you have:

Development Tools

- **Git** (2.0+) - Version control
- **Python** (3.8+) - Backend runtime
- **Node.js** (16+) & npm - Frontend build tools
- **MySQL Workbench** (optional) - Database management

Cloud Accounts (All Free Tier)

- **GitHub** account - Code repository
- **Railway** account - MySQL database hosting
- **Render** account - Backend API hosting
- **Netlify** account - Frontend hosting

Local Environment

- Terminal/Command Line access
 - Text editor or IDE (VS Code recommended)
 - Web browser (Chrome, Firefox, Safari)
-

Database Setup (Railway)

Step 1: Create MySQL Database

1. Go to railway.app
2. Sign up/login with GitHub
3. Click "New Project"
4. Select "Provision MySQL"
5. Wait for database to provision (~30 seconds)

Step 2: Get Database Credentials

1. Click on your MySQL service
2. Go to "Variables" tab
3. Copy these values:

```
bash
```

```
MYSQLHOST      = viaduct.proxy.rlwy.net
MYSQLPORT      = 12345
MYSQLUSER      = root
MYSQLPASSWORD  = your_password_here
MYSQLDATABASE  = railway
```

Important: Note the PORT

Step 3: Connect to Database

Using MySQL command line:

```
bash
```

```
mysql -h viaduct.proxy.rlwy.net -P 12345 -u root -p railway
# Enter password when prompted
```

Or using MySQL Workbench:

- Hostname: `viaduct.proxy.rlwy.net`
- Port: `12345` (your actual port)
- Username: `root`
- Password: (your Railway password)

Step 4: Load Schema and Data

```
bash
```

```
# Clone the repository first
git clone https://github.com/yourusername/CSCE548-BudgetingApp.git
cd CSCE548-BudgetingApp/backend

# Load schema
mysql -h viaduct.proxy.rlwy.net -P 12345 -u root -p railway < schema.sql

# Load test data
mysql -h viaduct.proxy.rlwy.net -P 12345 -u root -p railway < test_data.sql
```

Step 5: Verify Database

```
sql
```

```
-- Connect to database
mysql -h viaduct.proxy.rlwy.net -P 12345 -u root -p railway

-- Check tables
SHOW TABLES;

-- Should show: users, categories, budgets, budget_rules, transactions

-- Check data
SELECT COUNT(*) FROM users;
SELECT COUNT(*) FROM transactions;

-- Should show 143 total rows across tables
```

Database setup complete!

Backend Deployment (Render)

Step 1: Prepare Backend Files

Ensure your repository has these files in `backend/`:

```
backend/
— models/           # Data Access Layer
— business/         # Business Layer
— app.py            # Flask service
— db_config.py      # Database connection
— requirements_service.txt
— Procfile
— .env (local only - not in git)
```

Verify `requirements_service.txt`:

```
Flask==3.0.0
Flask-CORS==4.0.0
mysql-connector-python==8.2.0
python-dotenv==1.0.0
gunicorn==21.2.0
```

Verify `Procfile`:

```
web: gunicorn app:app
```

Step 2: Create Render Web Service

1. Go to render.com
2. Sign up/login with GitHub
3. Click "New +" → "Web Service"
4. Connect your GitHub repository
5. Select `CSCE548-BudgetingApp`

Step 3: Configure Build Settings

Name:	budget-tracker-api
Region:	Oregon (or closest)
Branch:	main
Root Directory:	backend
Runtime:	Python 3
Build Command:	<code>pip install -r requirements_service.txt</code>
Start Command:	<code>gunicorn app:app</code>

Step 4: Add Environment Variables

Click "Advanced" → "Add Environment Variable"

Add all Railway database credentials:

DB_HOST	=	viaduct.proxy.rlwy.net
DB_PORT	=	12345
DB_USER	=	root
DB_PASSWORD	=	your_railway_password
DB_NAME	=	railway

Step 5: Deploy

1. Click "Create Web Service"
2. Wait 5-10 minutes for first deployment
3. Watch build logs for errors
4. Once deployed, note your URL: `https://your-app.onrender.com`

Step 6: Test Backend API

```
bash

# Test health endpoint
curl https://your-app.onrender.com/

# Should return:
{
  "service": "Budget Tracker API",
  "status": "running",
  "version": "1.0.0"
}

# Test users endpoint
curl https://your-app.onrender.com/api/users

# Should return JSON array of users
```

Backend deployment complete!

Note: Render free tier spins down after inactivity. First request may take 30-60 seconds.

Frontend Deployment (Netlify)

Step 1: Prepare Frontend Files

Ensure your repository has these files in `frontend/`:

```
frontend/  
— public/  
|   — index.html  
— src/  
|   — components/  
|   |   — Users.jsx  
|   |   — Categories.jsx  
|   |   — Budgets.jsx  
|   |   — BudgetRules.jsx  
|   |   — Transactions.jsx  
|   — services/  
|   |   — api.js  
|   — App.jsx  
|   — App.css  
|   — index.js  
|   — index.css  
— package.json
```

Verify `package.json` has build script:

```
json  
  
"scripts": {  
  "start": "react-scripts start",  
  "build": "CI=false react-scripts build",  
  "test": "react-scripts test",  
  "eject": "react-scripts eject"  
}
```

Step 2: Create Netlify Site

1. Go to netlify.com
2. Sign up/login with GitHub
3. Click "Add new site" → "Import an existing project"
4. Choose **GitHub**
5. Select `CSCE548-BudgetingApp` repository

Step 3: Configure Build Settings

```
Base directory:    frontend  
Build command:    npm run build
```

Publish directory: frontend/build

Step 4: Add Environment Variable

1. Before deploying, click "**Show advanced**"
2. Click "**New variable**"
3. Add:

Key: REACT_APP_API_URL

Value: `https://your-backend.onrender.com/api`

Important: Use YOUR Render backend URL + `/api`

Step 5: Deploy

1. Click "**Deploy site**"
2. Wait 3-5 minutes for build
3. Once deployed, note your URL: `https://random-name.netlify.app`

Step 6: Customize Domain (Optional)

1. Go to **Site settings** → **Domain management**
2. Click "**Options**" → "**Edit site name**"
3. Change to: `budget-tracker-yourname`
4. New URL: `https://budget-tracker-yourname.netlify.app`

Step 7: Test Frontend

1. Visit your Netlify URL
2. Should see purple-themed Budget Tracker app
3. Navigate to each section:
 - Users
 - Categories
 - Budgets
 - Budget Rules
 - Transactions

Frontend deployment complete!

Local Development Setup

For local testing and development:

Step 1: Clone Repository

```
bash

git clone https://github.com/yourusername/CSCE548-BudgetingApp.git
cd CSCE548-BudgetingApp
```

Step 2: Setup Backend

```
bash

cd backend

# Install dependencies
pip install -r requirements_service.txt --break-system-packages

# Create .env file with Railway credentials
cat > .env << EOF
DB_HOST=viaduct.proxy.rlwy.net
DB_PORT=12345
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=railway
EOF

# Start Flask server
python app.py

# Should show: Running on http://127.0.0.1:5001
```

Step 3: Setup Frontend (New Terminal)

```
bash
```

```
cd frontend

# Install dependencies
npm install

# Start React dev server
npm start

# Should open browser at http://localhost:3000
```

Step 4: Test Locally

- Backend: <http://localhost:5001>
- Frontend: <http://localhost:3000>
- Both should be running simultaneously

Local development setup complete!

Testing & Verification

Full System Test Checklist

Test each operation for all 5 tables:

Users Table

- ☐ Create new user
- ☐ View all users
- ☐ View single user
- ☐ Update user
- ☐ Delete user

Categories Table

- ☐ Create new category
- ☐ View all categories
- ☐ View single category
- ☐ Update category
- ☐ Delete category

Budgets Table

- ☐ Create new budget
- ☐ View all budgets
- ☐ View single budget
- ☐ **Filter budgets by user** (subset query)
- ☐ Update budget
- ☐ Delete budget

Budget Rules Table

- ☐ Create new budget rule
- ☐ View all rules
- ☐ View single rule
- ☐ **Filter rules by budget** (subset query)
- ☐ Update rule
- ☐ Delete rule

Transactions Table

- ☐ Create new transaction
- ☐ View all transactions
- ☐ View single transaction
- ☐ **Filter transactions by user** (subset query)
- ☐ Update transaction
- ☐ Delete transaction

Database Verification

After each operation, verify in Railway database:

```
sql
```

```
-- After creating a user
SELECT * FROM users ORDER BY created_at DESC LIMIT 1;

-- After updating a budget
SELECT * FROM budgets WHERE budget_id = 123;

-- After deleting a category
SELECT * FROM categories WHERE category_id = 456;
-- Should return empty

-- Verify subset queries
SELECT * FROM budgets WHERE user_id = 1;
SELECT * FROM budget_rules WHERE budget_id = 5;
SELECT * FROM transactions WHERE user_id = 2 LIMIT 10;
```

Expected Results

All operations should:

- Display success message in UI
 - Update database immediately
 - Reflect changes in subsequent queries
 - Handle errors gracefully
 - Validate business rules
-

AI-Generated Code Analysis

AI Tool Used

Primary Tool: Claude (Anthropic)

Model: Claude Sonnet 4.5

Usage: All code generation, debugging, and deployment guidance

Prompts Used Per Layer

Project 1: Data Layer

Initial Prompt(s)

Initially, I used two prompts to generate the first version of my code; the first one was to verify that it was possible to create my budgeting app, and the second one was to actually generate the

code:

First Prompt:

NOTE: I attached a PDF version of the instructions to this message 'The goal of this class is to eventually create a full stack application, today we will be starting on the data layer. First I want to brainstorm about how this may be done. My current idea is for my application to be a budgeting app however I am not sure if I can allow it to fulfill the requirements. I am a graduate student so I must follow the additional outlined requirements for a graduate student. Because we are only making the data layer right now and this chat is theoretical meaning there will be no code yet I won't give too many specifics. The app should allow users to enter and track their spendings it should have several different budget models (moderate, strict) and the user may be able to create their own. is it possible to fulfill the requirements and create this app. we will also be using python and mysql to create this.'

Second Prompt:

'Alright let's begin generating the data layer, we will leave the overview document for later'

Issues and Roadblocks After the Initial Prompt(s) (i.e. Issues I was forced to fix)

Because of Claude's effectiveness in coding, I found that really nothing went wrong with the project. All of the CRUD operations were there and there were 5 tables. However, the foreign key relationships were not showing up within the MySQL workbench 'inspection' tab. To fix this, I initially examined the schema of the database to see if there was anything wrong with and ran some queries to try and see if the foreign keys would show up; the foreign keys were showing up within the codebase, however, they were not showing up in MySQL workbench. I began troubleshooting some more by uninstalling the application and refreshing the tables, until I concluded that it may be a version mismatch between the MySQL installation I have on my computer and the MySQL workbench application. This theory was further enhanced by the EER document created by MySQL showing the foreign keys.

I would not necessarily call this an issue that I was forced to fix because it was not caused by the AI, it was due to a version mismatch. Because I can always query and view the foreign keys, and I've implemented code to verify the foreign keys (more on that later), it is not that detrimental to me, so I have decided to keep the installations as is.

Changes After the Initial Implementation

Most notably, I added code to validate the creation of the foreign key relationships. This was added as a result to the issue outlined above. There were also some minor changes to the configurations of the database, like the imports (because I prefer more granular imports than having them on one line). Additionally, I added an environment variable to store the password for the root user within MySQL titled 'SQLPASS'. Lastly, a change I made was adding a requirements document, just so the correct dependencies/frameworks can be installed in case I was

working on another machine (though this is not reflected in the first push on github); this also allows me to keep track of all of the dependencies/frameworks I add.

The code for the schema changes looks like this:

```
-- Verify foreign keys were created
SELECT
    'Foreign Keys Verification' AS info,
    COUNT(*) as total_foreign_keys
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'budget_tracker'
AND REFERENCED_TABLE_NAME IS NOT NULL;

-- Show all foreign keys
SELECT
    TABLE_NAME,
    COLUMN_NAME,
    CONSTRAINT_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'budget_tracker'
AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME;
```

AI Effectiveness (i.e. What did it do Well?)

Overall, I would rate Claude - ver. Sonnet 4.5 **highly effective**. The initial code base it provided was very good. It had zero major errors; it implemented the code cleanly as the instructions required. The only error I really had was a installation version mismatch (outlined above), and that was an error on my end. The front-end, foreign keys, tables, CRUD, test data ... were all implemented properly.

In short, the AI model used was highly effective, it did not miss anything, and there were no major errors I was forced to solve

Project 2: Business & Service Layers

Initial Prompt(s)

Like last time, I initially used two prompts to generate the first version of my code; the first one was to verify that it was possible to create my budgeting app, and the second one was to actually generate the code. I found that this method was very effective last time, so I decided to use it again, however, I used more prompts to segment the project into parts:

NOTE: I attached a PDF version of the instructions to this prompt. It is also important to acknowledge that this is within the same chat that the initial business layer was in to maintain the continuity for my application:

'Now we are going to move into the second project, the bussiness layer.'

Before we begin coding, I want you to help me find a list of services and microservices that I could use and how I would host them. Attached are the instructions, remeber, I am a graduate student and would be following graduate requirements. I can send an explanation describing the bussiness and service layers to you if you need more info'

Second Prompt:

NOTE: I attached a pdf document containing condensed information about the business and service layers; to summarize, the document stated that the business layer was to be treated as a manager for the code and that the service layer would host these services on an API endpoint

'This is the information pertinent to the bussiness and service layer. Let's take this 1 at a time. We will start with the code of the bussiness layer.'

Third and Final Prompt

'Ok let's get started on the service layer and the deployment. What services do we currently have that we are hosting'

With these three prompts I had the initial localhost version of the business and service layer

Issues and Roadblocks After the Initial Prompt(s) (i.e. Issues I was forced to fix)

Claude did a great job when it came to generating the initial versions of the code that it ran locally, but it fell short when it came to transitioning it over into the hosted portion of the project. This was a really big headache because the AI model was incapable of providing the proper instructions to get the services hosted, so I had to figure out how to do it myself. I used railway to host my database and render to host the API

Railway caused the most problems with the deployment. The main issue was connecting to my public instance of railway, and not the local one. Through about two combined hours of railway, I was able to take it upon myself and find the solution to connecting my database to railway. The render.com setup for hosting the API was very simple, so I did not need the AI's help there.

Changes After the Inital Implementation

After the initial implementation. I added the functionality for connecting my services and API to the hosted site; this was partially AI assited, though the connection methodology was figured out by me (as stated above). I also ended up creating a script that would test the connection using AI. that file is called 'test_railway_connection.py'

More specifically, I had to change the schema to point towards railway and render, as oppsed to local host.

AI Effectiveness (i.e. What did it do Well?)

For this specific project I would rate the AI **3/5 somewhat effective**. The initial code base it provided was very good, however, it offered little to no help in switching that local instance into a hosted instance. The code it did write initially was very functional though, with that having no errors.

Project 3: Frontend

Initial Prompt(s)

My usual methodology for using the Claude AI is to segment the process through the use of multiple prompts. I have adopted a similar structure here. My initial prompt was to identify the scope of the project to see whether or not it was feasible and to find possible methods of execution for the project, and the following prompt or prompts is to actually gain the code.

Prompt 1:

NOTE: I attached a PDF version of the instructions to this prompt. It is also important to acknowledge that, like last time, this is within the same chat that all of the projects were created in.

‘OK, let's get started on the next project. Attached are the instructions. I want you to tell me how we might go about its implementation as well as its frameworks’

In response to Prompt 1, I decided to use react and netlify to complete this edition of the project.

Prompt 2:

NOTE: I attached multiple .css files from my capstone computing project, a web app with a style that I am fond of. However, I wanted the color scheme to be primarily purple instead of primarily garnet.

NOTE: I am referring to a file structure outlined by Claude. It gave a large amount of files which are easily generated through the creation of a react project. When I said “is there a way to get alot of these files you've outlined here?”, I was referring to this large file structure. It is also important to note that this is the very first time I have used react.

‘Ok let's get started with the code. we can use react and netlify, but we can do deployment later. give the updated file locations too. Let's build it in chunks because it will be alot. with react, how do I install it? and is there a way to get alot of these files you've outlined here? . I have attached files to aid in the appearance to use as reference. the color scheme has garnet, but I want mine to be purple (#A020F0)’

Issues and Roadblocks After the Initial Prompt(s) (i.e. Issues I was forced to fix)

Per-usual, Claude was very effective in generating the initial code and giving instructions to do

so. The very first version was very visually pleasing, and the instructions were very clear. However, there were several features that were incorrectly implemented that may have been an oversight in the last project. It had to do with how the front-end was interpreting the input of date ranges on the external site. This was the error:

```
jsx  
  
**Failed: Database error: '<=' not supported between instances of 'str' and 'int'"
```

This error was seen in the `budget_manager.py`, `budget_rule_manager.py`, and `transaction.py` files. It made it better that it was consistent across multiple files, so it was easier to debug. The primary error was that numeric values were being seen as strings, or the other way around. To fix this, I had to add input normalization in the effected files, explicitly type casting the variables so there were no discrepancies. As stated before, this was caused by how the front-end was interacting with the back-end, and this took a large amount of development time to complete.

I also had a few other small errors within the deployment phase of this project. They were primarily user-errors on my side. The most notable was forgetting to add “/api” to the URL when connecting my netlify instance to my API instance hosted on Render.

Changes After the Inital Implementation

After the inital implementation, I changed the `budget_manager.py`, `budget_rule_manager.py`, and `transaction.py` files to combat the major error I outlined above. The initial implementation already had the potential for connection to external hosted services, so I did not need to make modifications there; however, I did have to go and make the connections physically myself (specifically connecting netlify to my API instance hosted on Render).

AI Effectiveness (i.e. What did it do Well?)

For this specific project I would rate the AI **3.5/5 somewhat effective**. The initial code base it provided was very good, and it gave phenominal instructions on how to install npm and react. It did however create bugs that were consistent across multiple files; it was not able to solve these bugs on its own seemingly because it looked over them, but I was able to fix them. Due to the consistency of the bugs, I did not take too many points off. Everything but those bugs were implemented properly, and it styled the project very well.

Overall Assessment

AI Effectiveness Score: 9/10

Strengths:

- Generated 95% of functional code correctly
- Saved approximately 40-50 hours of manual coding

- Produced professional, industry-standard architecture
- Excellent at explaining concepts and debugging

Weaknesses:

- Required manual intervention for deployment edge cases
- Needed debugging for type conversion issues
- Some platform-specific knowledge gaps

Conclusion:

AI was highly effective for rapid prototyping and development. Most of the issues encountered were minor and quickly resolved with targeted prompts or manual fixes. The tool excelled at generating boilerplate code, implementing patterns, and providing deployment guidance.

Troubleshooting

Common Issues

Backend Issues

Issue: `Database connection failed`

- Check Railway credentials in Render environment variables
- Verify Railway MySQL is running (green status)
- Confirm port is correct (not 3306)

Issue: `Render app spins down`

- Free tier sleeps after 15 minutes inactivity
- First request takes 30-60 seconds to wake
- Upgrade to paid tier for always-on service

Issue: `Module not found` during Render build

- Verify `requirements_service.txt` is in backend/ folder
- Check build command includes `-r requirements_service.txt`
- Ensure Python version compatibility

Frontend Issues

Issue: `Cannot connect to API`

- Verify `REACT_APP_API_URL` is set correctly in Netlify
- Check URL ends with `/api`
- Confirm backend is deployed and running
- Check browser console for CORS errors

Issue: `Build failed on Netlify`

- Check deploy logs for specific error
- Verify `package.json` has `build` script
- Try `CI=false` to ignore warnings
- Ensure `frontend/build` publish directory is correct

Issue: `Shows React logo instead of app`

- Build deployed from wrong directory
- Check base directory is `frontend`
- Verify GitHub has latest code
- Try "Clear cache and redeploy"

Database Issues

Issue: `Foreign keys not showing in MySQL Workbench`

- Known bug with MySQL Workbench 8.x and MySQL 9.x
- Verify via SQL: `SHOW CREATE TABLE table_name;`
- Relationships exist even if GUI doesn't show them

Issue: `Cannot connect to Railway database`

- Check you're using the PUBLIC hostname (ends with `.proxy.rlwy.net`)
- Don't use `mysql.railway.internal` (internal only)
- Verify port is correct
- Check Railway service is running

Local Development Issues

Issue: `Port 3000/5001 already in use`

```
bash
```

```
# Kill process on port 3000
lsof -ti:3000 | xargs kill -9

# Kill process on port 5001
lsof -ti:5001 | xargs kill -9
```

Issue: `pip install fails`

- Use `--break-system-packages` flag on newer Python versions
 - Or create virtual environment: `python -m venv venv`
-

Deployment Verification Checklist

Use this checklist to verify successful deployment:

Database (Railway)

- ☐ MySQL service is running (green status)
- ☐ Can connect via MySQL Workbench
- ☐ All 5 tables exist
- ☐ Tables have data (143 rows total)
- ☐ Foreign key relationships are present

Backend (Render)

- ☐ Web service is deployed and running
- ☐ Health endpoint returns JSON: `https://your-app.onrender.com/`
- ☐ API endpoints accessible: `https://your-app.onrender.com/api/users`
- ☐ Environment variables are set (5 DB credentials)
- ☐ Build logs show no errors

Frontend (Netlify)

- ☐ Site is deployed and live
- ☐ Purple theme displays correctly
- ☐ All 5 navigation links work
- ☐ Environment variable `REACT_APP_API_URL` is set
- ☐ No console errors in browser (F12)

Full System

- ☐ Can create records in all 5 tables
- ☐ Can view all records
- ☐ Can view single records
- ☐ Can update records
- ☐ Can delete records
- ☐ Subset queries work (filter dropdowns)
- ☐ Data persists across sessions
- ☐ Changes reflect immediately in UI

If all items are checked, deployment is successful!

Project Statistics

Code Metrics

- **Total Files:** 25
- **Lines of Code:** ~3,500
- **Database Tables:** 5
- **Foreign Keys:** 5
- **API Endpoints:** 25
- **React Components:** 5
- **Test Data Rows:** 143

Deployment URLs

- **Frontend:** <https://benevolent-nasturtium-26b5e7.netlify.app>
 - **Backend:** <https://csce548-budgetingapp.onrender.com>
 - **Database:** Railway (private)
 - **Repository:** <https://github.com/Juliusp36/CSCE548-BudgetingApp>
-

Credits

Course: CSCE 548 - Full Stack Development

Institution: University of South Carolina

Semester: Spring 2026

Technologies: React, Flask, Python, MySQL, Railway, Render, Netlify

AI Assistance: Claude (Anthropic) for code generation

License

Academic project for CSCE 548. All rights reserved.

Last Updated: March 13, 2026

Version: 1.0.0

Status: Complete and Deployed